Ejercicio: Viaje por carretera

Estás haciendo un viaje por carretera de D **kilómetros** en línea recta. A lo largo del camino hay **gasolineras** en posiciones específicas (en kilómetros desde el inicio), dadas en una lista estaciones ordenada de menor a mayor.

Tu auto tiene un tanque con autonomía de A **kilómetros**. Empiezas con el tanque **lleno** en el kilómetro 0.

En cada gasolinera puedes decidir **cuántos litros cargar**, pero por simplicidad, cada gasolinera ofrece **tres opciones de carga**:

Opción	Litros	Costo
Carga pequeña	`litros[0]`	`costos[0]`
Carga media	`litros[1]`	`costos[1]`
Carga grande	`litros[2]`	`costos[2]`

Cada litro te da **1 km** de autonomía. El tanque tiene capacidad máxima de A **litros** (no puedes pasarte).

Debes llegar al kilómetro D. Si en algún momento te quedas sin gasolina antes de llegar, el viaje es imposible.

Devuelve el **costo mínimo** para llegar a destino. Si es imposible, devuelve -1.

Ejemplo 1
D = 100
A = 50
estaciones = [20, 50, 80]
litros = [10, 25, 40]
costos = [5, 12, 18]

Cada gasolinera ofrece **tres opciones de carga**, *tu puedes elegir solo* **una sola opción por estación**. La diferencia es cuál opción elegís en cada una, y eso solo se ve comparando el costo **total**, no estación por estación.

Salida: 23

Explicación:

- Empiezas con 50 litros (autonomía hasta km 50).
- Y eso me alcanza para llegar con 0 al km 50.
- En km 50: como estoy en 0, cargo 40 para llegar 80, porque con 10 o 25 no me da.
- En km 80, me queda solo 10, entonce nada mas requiero 10, para llegar a 100
- Llegas a km 100 con +0 L

Punto	Llego con	Decisión	Salgo con	Costo
km 0	50 L (tanque lleno)	—	50 L	—
km 20	30 L	**no cargo** (0 L)	30 L	$0
km 50	**0 L**	+40 L (opción grande)	40 L	$18
km 80	**10 L**	+10 L (opción	20 L	$5

Punto	Llego con	Decisión	Salgo con	Costo
		pequeña)		
km 100	**0 L** ✅	—	—	**Total $23**

Ejemplo 2
D = 200
A = 100
estaciones = [50, 120]
litros = [20, 50, 90]
costos = [8, 18, 30]
Salida: -1
Explicación:
•Empiezas con 100 L (autonomía hasta km 100).
•La siguiente estación está en km 50 → llegas con 50 L.
•Pero de km 50 a km 120 hay 70 km, y solo tienes 50 L + lo que cargues.
•Si cargas 90 L → 140 L, topado a 100 L → llegas a km 120 con 30 L.
•De km 120 a km 200 hay 80 km, pero solo tienes 30 L + lo que cargues.
•Si cargas 90 L → 120 L, topado a 100 L → llegas a km 200 con 20 L. ¡Sí se puede!
Espera, entonces **sí** se puede. ¿Por qué la salida es -1? 🤔
(Pista: revisa bien las distancias. ¿Hay algún tramo entre estaciones que supere la autonomía máxima?)

Ejemplo 3
D = 150
A = 60
estaciones = [30, 70, 110]
litros = [15, 30, 50]
costos = [6, 11, 17]
Salida: 28
Explicación:
•Empiezas con 60 L (autonomía hasta km 60).
•Km 30: llegas con 30 L. Cargas 30 L → 60 L. Costo: 11.
•Km 70: llegas con 20 L. Cargas 30 L → 50 L. Costo: 11.
•Km 110: llegas con 10 L. Cargas 15 L → 25 L. Costo: 6. Total: 28.
•Llegas a km 150 con 25 - 40 = -15 L... ¡imposible!
(Pista: hay que cargar más en algún punto. La respuesta es 28, ¿puedes encontrar la combinación correcta?)

🧭 Tu misión

Antes de escribir código, responde en papel:
1.**¿Cuál es el estado?**
•Pista: no basta con "en qué estación estoy". También importa **cuánta gasolina me queda**.
•Este es un estado **bidimensional**: (índice_estación, gasolina_restante).
2.**¿Qué decisiones puedes tomar desde ese estado?**
•Tres opciones de carga (o quizás también "no cargar").
3.**Para cada decisión: ¿cuánto cuesta y a qué nuevo estado te lleva?**

•El nuevo estado depende de **cuánta gasolina gastas** en llegar a la siguiente estación.
4.**¿Qué quieres?** (min, max, contar...)
5.**Escribe la recurrencia.**
6.**¿Cuáles son los casos base?**
•Llegar a destino → costo 0.
•Quedarte sin gasolina antes de llegar → imposible (∞).
•No poder llegar a la siguiente estación → imposible.

🔍 Pista clave

Este problema introduce **dos novedades** respecto a los anteriores:
1.**Estado bidimensional:** no basta con "dónde estoy", también importa "con cuánto llego".
2.**Restricción de capacidad:** no puedes cargar más de `A` litros.
Pero la **estructura es la misma**: estado + opciones + costo + nuevo estado.

🎯 Comparación con los anteriores

Problema	Estado	Opciones	Operación
Dados	(n dados, suma)	cara 1..K	suma
Tickets	día `d`	pase 1, 7, 30	min
Pizzas	índice pedido	bicicleta, moto, auto	min
Muro	metros restantes	bloque 1, 2, 3	min
Vacaciones	día `d`	paquete 1d, 7d, 30d	min
Gasolina	**(estación, gasolina)**	**carga S, M, L**	**min**

¿Ves cómo el patrón se repite una y otra vez? Eso es lo que debes internalizar.

```
int f(int num_estacion, int gasolina_actual)
```

num_estacion: En la estación que estoy parado actualmente, que incluso puede ser en el inicio que vale 0, porq no estoy en una gasolinera apenas voy a empezar a tanque lleno
gasolina_actual: La cantidad de gasolina que tengo actualmente, cuando comienza el tanque esta lleno.

Caso base

```
if (num_estacion == destino) return 0;  //Llegamos al num_estacion de destino (al final 100)
```

Memorización

```
if (memo[num_estacion][gasolina_actual] != -1) return memo[num_estacion][gasolina_actual];
```

Solo **avanza si la gasolina actual alcanza para el tramo**, y no suma ningún costo (costos[k]). Es una decisión de **"puedo permitirme no cargar"**. Recursión sin sumar costo.

```
if (gasolina_actual >= distancia_Prox_Estacion)  //"si me alcanza sin cargar, avanzá sin pagar".
    mejor = min(mejor, f(num_estacion + 1, gasolina_actual – distancia_Prox_Estacion));
```

- **No hay** costos[k] + ... → **no pagás nada** → **no cargaste**.
- **gasolina_actual >= distancia_Prox_Estacion** es solo un **filtro de factibilidad**: "si no cargo, ¿llego?".
- **gasolina_actual - distancia_Prox_Estacion** es la gasolina con la que **llegás** a la próxima estación(**num_estacion + 1**) habiendo gastado el tramo.

Nota:

- **gasolina_actual >= distancia_Prox_Estacion** se lee: "¿la gasolina actual alcanza para el tramo?"
- **gasolina_actual – distancia_Prox_Estacion)** se lee: "con cuánto llego a la próxima estación si no cargo".

Este for representa la decisión de **cargar combustible en la estación actual**, probando cada una de las **opciones de carga disponibles (k = 0, 1, 2)**. Es decir, en vez de un único "cargar sí/no", hay **tres formas distintas de cargar** (cada una con su litraje y su costo), y el algoritmo prueba las tres.

Cada **k** es una **alternativa de carga** con:

- **litros[k]** → cuántos litros agregás
- **costos[k]** → cuánto pagás por esa carga

El for es para evaluar las tres y quedarte con la mejor. Cada iteración es un "mundo hipotético" distinto.

```
// Opciones 2..4: cargar una de las tres opciones
for (int k = 0; k < m_lit; k++) {  //Opciones de (k = 0, 1, 2)
    int gasolina_tras_carga = min(A, gasolina_actual + litros[k]); // capacidad máxima A

  if (gasolina_tras_carga  >= distancia_Prox_Estacion) // ¿alcanza para el tramo?
     mejor = min(mejor, costos[k] + f(num_estacion + 1, gasolina_tras_carga  - distancia_Prox_Estacion));
}
```

```
int nuevo = min(A, gasolina_actual + litros[k]);
```

Acá simulás la carga:

- **gasolina_actual + litros[k]**→ lo que tendrías si agregás litros[k] sin límite.
- **min(A, ...)** → pero el **tanque tiene capacidad máxima A**, así que **no podés pasarte**. **Se lee**: "cargo, pero sin pasarme del tanque".
- **Que significa A Pagás el costo completo igual** (pagás por la carga aunque se derrame). Lo único que hace es **recortar la gasolina que queda en el tanque para no superar la capacidad A**, es decir, si la opción no entra entera, solo "agarrás" la parte que cabe (el excedente se pierde) pero igual pagás todo; pero no pagás menos por eso.

if (gasolina_tras_carga >= distancia_Prox_Estacion) : Filtro de factibilidad: después de cargar, ¿tienés suficiente para llegar a la próxima estación?

- Si **gasolina_tras_carga < distancia_Prox_Estacion** → esta **carga no sirve**, no podés avanzar. Se descarta (no se ejecuta el cuerpo del if).
- Si **gasolina_tras_carga >= distancia_Prox_Estacion** → esta **carga es viable**, se evalúa el costo.

Ojo: este if **no decide cargar**; decide si esta carga particular te permite avanzar. La decisión de cargar ya está implícita en que estás dentro del for y vas a sumar costos[k].

```
mejor = min(mejor, costos[k] + f(num_estacion + 1, gasolina_tras_carga  - distancia_Prox_Estacion));
```

Acá está el corazón. Se lee:

- **gasolina_tras_carga - distancia_Prox_Estacion se lee:** "con cuánto llego a la próxima estación después de cargar y consumir el tramo".
- **costos[k] + f(...) : se lee** "pago esta carga + lo que cueste el resto del viaje".

```cpp
#include <bits/stdc++.h>
using namespace std;

int D, A, m_lit,destino;
vector<int> pos; // pos[0]=0, pos[1..m]=estaciones, pos[m+1]=D
vector<int> litros, costos;
vector<vector<int>> memo; // memo[num_estacion][gas]
const int INF = 1e9;

//comienzo num_estacion 0, tanque lleno
int f(int num_estacion, int gasolina_actual) {
  if (num_estacion == destino) return 0; // llegamos al num_estacion de destino final 100
  if (memo[num_estacion][gasolina_actual] != -1) return memo[num_estacion][gasolina_actual];

   //0,20,50,80,100 es la distancia_Prox_Estacionancia a la siguiente estación (o al destino).
   int distancia_Prox_Estacion = pos[num_estacion + 1] - pos[num_estacion]; //20-0, 50-20, 80-50, 100-80
   int mejor = INF;
   // "si me alcanza sin cargar, avanzá sin pagar".
  if (gasolina_actual >= distancia_Prox_Estacion)
     mejor = min(mejor, f(num_estacion + 1, gasolina_actual - distancia_Prox_Estacion));

  // Sino alcanza tengo que cargar una de las tres opciones
  for (int k = 0; k < m_lit; k++) {
       int gasolina_tras_carga = min(A, gasolina_actual + litros[k]);
       if (gasolina_tras_carga >= distancia_Prox_Estacion) // ¿alcanza para el tramo?
           //con cuánto llego a la próxima estación después de cargar y consumir el tramo
          mejor = min(mejor, costos[k] + f(num_estacion + 1, gasolina_tras_carga - distancia_Prox_Estacion));
  }

  return memo[num_estacion][gasolina_actual] = mejor;
}



int main() {
//D = 100; A = 50;
//vector<int> estaciones = {20, 50, 80}; //están las gasolineras reales, sin inicio ni destino
// litros = {10, 25, 40};
//costos = {5, 12, 18}; //23

D = 150; A = 60;
vector<int> estaciones = {30, 70, 110};
litros = {15, 30, 50};
costos = {6, 11, 17}; //11+11+11=33

m_lit = litros.size();

//estructura de la DP: todas las posiciones donde hay un estado:
//**inicio, estaciones, destino**.
```

```
//pos= {0,20, 50, 80,100}
pos.clear();
pos.push_back(0); //inicio
for (int e : estaciones) pos.push_back(e); //estaciones
pos.push_back(D); // destino final
destino = (int)pos.size() - 1;

memo.assign(pos.size(), vector<int>(A + 1, -1));

int res = f(0, A); // comienzo num_estacion 0, tanque lleno
cout << "Costo mínimo: " << (res == INF ? -1 : res) << endl; // 23
}
```